

Q1. write a code to reverse the elements of stack and then find the index of minimum valued number using array.

```
⇒ #include <stdio.h>
#define TRUE 1
#define FALSE 0
```

```
// declaring the functions to be used.
```

```
void Push(int);
int Pop();
void PrintStack();
void reverse();
void MinElement();
```

```
// declare stack
```

```
int top = -1;
int arr_stack[20];
int SIZE = 0;
```

```
int main(){
```

```
    int temp; // for scanning the input
```

```
    // inserting elements in stack
```

```
    printf("Enter the number of elements (not more than 20): ");
```

```
    scanf("%d", &SIZE);
```

```
    printf("Enter the elements of stack:\n");
```

```
    for (int i=0; i<SIZE; i++){
```

```
        scanf("%d", &temp);
```

```
        Push(temp);
```

```
    }
```

```
    // displaying the originally inserted value from stack
```

```
    printf("original stack\n");
```

```
    PrintStack();
```

Name - Aniket Kumar

Page 2 of 12

Registration no. - 12108348

Q.No. - 1

Roll no. - B62

reverse(); // for reversing the stack elements.

// displaying reversed stack

printf("Reversed stack\n");

PrintStack();

// displaying minimum valued number's index

MinElement();

return 0;

}

// function definitions

// for checking if stack is full or not

int isfull(){

if (top >= SIZE-1)

return TRUE;

else

return FALSE;

}

// checks if stack is empty or not

int isEmpty(){

if (top == -1)

return TRUE;

else

return FALSE;

}

// inserting an element to the stack and increment top index.

void Push(int num){

if (isfull())

printf("stack is full...\n");

else {

top = top + 1;

Name - Aniket Kumar

Page 3 of 12

Registration no. - 12108348

Q.No. - 1

Roll no. - B62

```
    arr_stack[top] = num;
  }
}
```

// removing top element from stack and decrement top index.

```
int pop() {
    if (isEmpty())
        printf("Stack is Empty... \n");
    else {
        top = top - 1;
        return arr_stack[top + 1];
    }
}
```

// for displaying all the elements of stack

```
void printStack() {
    if (top == -1) {
        printf("Stack is Empty... \n");
    }
    else {
        for (int i = top; i >= 0; i--) {
            // start traversing from top element.
            printf("%d ", arr_stack[i]);
        }
        printf("\n");
    }
}
```

// inserting element to the bottom of the stack

```
void insertAtBottom(int item) {
    if (isEmpty()) {
        push(item);
    }
}
```

Name - Aniket Kumar

Page 4 of 12

Registration no. - 12108348

Q. no. - 1

Roll no. - B62

else {

// store top most element in another variable and recursively

// insert the rest of the stack using insertAtBottom

int top = Pop();

insertAtBottom(item);

// finally insert the top element again.

Push(top);

}

}

// reversing the stack elements.

void reverse() {

if (!isEmpty()) {

// Keep popping top element until stack is empty.

int top = Pop();

reverse();

// After removing, insert top element to the bottom

insertAtBottom(top);

}

}

void minElement() {

int index = 0;

for (int i = top; i >= 0; i--) {

if (arr_stack[i] < arr_stack[index]) {

index = i;

}

}

printf("Min minimum element is Present at index %d and is %d(th) element from the top.", index, top - index + 1);

}

Name - Aniket Kumar

Page 5 of 12

Registration no. - 12108348

Q. no. - 1

Roll no. B62

Output:

Enter the number of elements (not more than 20): 5

Enter the elements of stack:

32 45 67 12 56

original stack

56 12 67 45 32

Reversed stack

32 45 67 12 56

Minimum element is Present at index 1 and is 4(th) element from the top.

Name - Aniket Kumar

Page 6 of 12

Registration no. - 12108348

Qno - 2

Roll no. - 862

Q2 write a code to delete the minimum valued item among N numbers from a queue.

```
⇒ #include <stdio.h>
#define MAX_SIZE 100
```

```
void insert();
void delete_min();
void display();
```

```
int arr_queue[MAX_SIZE]; // declaring queue
int front=0, rear=0;
```

```
int main() {
```

```
    int n;
```

```
    // inserting queue elements
```

```
    printf("Enter the no of queue elements: ");
```

```
    scanf("%d", &n);
```

```
    for(int i=0; i<n; i++) {
        insert();
```

```
    }
```

```
    // deleting minimum valued item from queue and display
    // rest elements of the queue.
```

```
    delete_min();
```

```
    return 0;
```

```
}
```

// function for inserting elements into queue at the ~~front~~ rear.

```
for #
void insert() {
    int item;
```

Name - Aniket Kumar
Registration no. - 12108348
Roll no. - B62

Page 7 of 12
Q No - 2

```
if (rear == MAX_SIZE)
    printf("\n ### Queue reached max!");
else {
    printf("\n Enter the value to be inserted: ");
    scanf("%d", &item);
    printf("\n # Position: %d, Inserted value: %d", rear+1, item);
    arr_queue[rear++] = item;
} // insert element at rear and increment rear by 1.
}
```

// function to delete minimum valued item.

```
void delete_min() {
    if (front == rear)
        printf("\n ### Queue is Empty!");
    else {
        int min_element, index = 0;
        // displaying the queue before deleting.
        display();
        // finding the index of minimum valued item
        for (int i = front; i < rear; i++) {
            if (arr_queue[i] < arr_queue[index]) {
                index = i;
            }
        }
        // displaying minimum valued item.
        min_element = arr_queue[index];
        printf("\n minimum element is %d.\n", min_element);
        printf("deleting it.\n");
    }
}
```

Name - Aniket Kumar

Registration no. - 12108348

Roll no. - B62

Page 8 of 12

Q no. 2

// deleting the element

```
for (int i = index; i < rear; i++) {  
    arr_queue[i] = arr_queue[i+1];
```

```
}
```

rear = rear - 1; // decrement in rear after deleting.

display(); // displaying after deletion.

```
}
```

```
}
```

// function for displaying queue items.

```
void display() {
```

```
    printf("In ## Queue size: %d\n", rear);
```

```
    for (int i = front; i < rear; i++)
```

```
        printf("%d ", arr_queue[i]);
```

```
}
```

Output:

Enter the no of queue elements: 5

Enter the value to be inserted: 32

Position: 1, Inserted value: 32

Queue size: 5

32 45 67 12 56

Minimum element is 12

Deleting it..

Queue size: 4

32 45 67 56

Name- Aniket Kumar

Registration no.- 12108348

Roll no.- B62

Page 9 of 12

Q.No.- 3

Q3. write a code to insert a node in queue using linked list.

```
⇒ #include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node { // declaring queue using linked list.
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *front;
```

```
struct node *queue;
```

```
void insert();
```

```
void display();
```

```
void main() {
```

```
    int choice;
```

```
    while(1) {
```

```
        printf("In chose option ");
```

```
        printf("In 1. Insert an element In 2. Display the queue  
                In 0.Exit In");
```

```
        printf("In Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch(choice) {
```

```
            case 1:
```

```
                insert();
```

```
                break;
```

```
            case 2:
```

```
                display();
```

```
                break;
```

Name - Aniket Kumar

Page 10 of 12

Registration no. - 12108348

Ans. - 3

Roll no. - B62

Case 0:

exit(0);

break;

default:

printf("\nEnter valid choice!\n");

}

}

}

// function for inserting node in queue using linked list.

void Insert()

{
struct node *Ptr;

int item;

// dynamically allocating memory.

Ptr = (struct node *) malloc(sizeof(struct node));

if (Ptr == NULL) { // Queue is full

printf("\n overflow\n");

return;

}

else {

printf("\nEnter value: ");

scanf("%d", &item);

Ptr->data = item;

if (front == NULL) // inserting node in empty queue

{

front = Ptr;

rear = Ptr;

front->next = NULL;

rear->next = NULL;

}

Name - Aniket Kumar

Page 11 of 12

Registration no. - 12108348

Q.No. - 3

Roll no. - 862

else // inserting node next to Previous node

{

rear->next = Ptr;

rear = Ptr;

rear->next = NULL;

}

Printf("Inserted!\n");

}

}

// function to display the elements of queue.

void display() {

struct node *Ptr;

Ptr = front;

if (front == NULL)

Printf("\n Empty queue\n");

else {

Printf("values inside Queues are: ");

while (Ptr != NULL) {

Printf("%d ", Ptr->data);

Ptr = Ptr->next;

}

Printf("\n");

}

}

Output:

Queue operation.

1. Insert an element
2. Display the Queue.
0. Exit

Name - Aniket Kumar

Registration no. - 12108348

Roll no. - 862

Page 12 of 12

Q.No. - 3

Enter your choice: 1

Enter value: 43

Intersted!

...

...

Enter your choice: 2

values inside Queue are: 43 46

C > CA1 > C Set5Q1.c > main()

```
4
5 #include <stdio.h>
6
7 #define TRUE 1
8 #define FALSE 0
9
10 void push(int);
11 int pop();
12 void printStack();
13 void reverse();
14 void MinElement();
15
16 // Stack data structure
17 int top = -1;
18 int arr_stack[20];
19 int SIZE = 0;
20
21 int main()
22 {
23     int temp;
24     /* Inserting elements in stack */
25     printf("Enter the number of elements
(not more than 20): ");
26     scanf("%d", &SIZE);
27     printf("Enter the elements of stack:\n");
28     for(int i=0; i<SIZE; i++){
29         scanf("%d", &temp);
30         push(temp);
31     }
32     printf("Original Stack\n");
33     printStack();
34     reverse();
35     printf("\nReversed Stack\n");
36     printStack();
37     MinElement();
38     return 0;
39 }
40
```

C > CA1 > C Set5Q1.c > main()

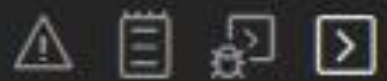
```
41 /*
42  Checks if Stack is Full or not
43 */
44 int isFull()
45 {
46     if (top >= SIZE - 1)
47         return TRUE;
48     else
49         return FALSE;
50 }
51
52 /*
53  Checks if Stack is Empty or not
54 */
55 int isEmpty()
56 {
57     if (top == -1)
58         return TRUE;
59     else
60         return FALSE;
61 }
62
63 /*
64  Adds an element to stack and then increment
top index
65 */
66 void push(int num)
67 {
68     if (isFull())
69         printf("Stack is Full...\n");
70     else
71     {
72         top = top + 1;
73         arr_stack[top] = num;
74     }
75 }
76
```

C > CA1 > C Set5Q1.c > main()

```
80 int pop()
81 {
82     if (isEmpty())
83         printf("Stack is Empty...\n");
84     else
85     {
86         top = top - 1;
87         return arr_stack[top + 1];
88     }
89 }
90
91 // Prints elements of stack
92 void printStack()
93 {
94     if (top == -1) // checks if stack is
95         empty
96     {
97         printf("Stack is empty\n");
98     }
99     else
100     {
101         for (int i = top; i >= 0; i--)
102         {
103             printf("%d ", arr_stack[i]);
104         }
105         printf("\n");
106     }
107
108 void insertAtBottom(int item)
109 {
110     if (isEmpty())
111     {
112         push(item);
113     }
114     else
115     {
116         /* Store the top most element of
```

C > CA1 > C Set5Q1.c > reverse()

```
122         bottom, push the
123         top element back to stack */
124         push(top);
125     }
126
127 void reverse()
128 {
129     if (!isEmpty())
130     {
131         /* keep on popping top element of
132         stack in
133         every recursive call till stack is
134         empty */
135         int top = pop();
136         reverse();
137
138         /* Now, insert the top element at
139         the bottom of stack */
140         insertAtBottom(top);
141     }
142
143 // for finding the index of min element
144 void MinElement()
145 {
146     int index = 0;
147     for (int i = top; i >= 0; i--)
148     {
149         if (arr_stack[i] < arr_stack[index])
150         {
151             index = i;
152         }
153     }
154     printf("\nMinimum element is present at
155     index %d and is %d(th) element from the
156     top.\n", index, top-index+1);
157 }
```



Enter the number of elements (not more than 20): 5

Enter the elements of stack:

32 45 67 12 56

Original Stack

56 12 67 45 32

Reversed Stack

32 45 67 12 56

Minimum element is present at index 1 and is 4(th)
element from the top.

[crypticani@fedora DSA_Programs]\$

```

C > CA1 > C Set5Q2.c > main()
1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4
5  void insert();
6  void delete_min();
7  void display();
8
9  int arr_queue[MAX_SIZE];
10 int rear = 0, front = 0;
11
12 int main()
13 {
14     int n;
15     // queue operations for enqueueing elements,
16     printf("Enter the no of queue elements: ");
17     scanf("%d", &n);
18     for(int i=0; i<n; i++){
19         insert();
20     }
21     // deleting minimum valued item from queue and
22     // displaying the elements of queue
23     delete_min();
24     return 0;
25 }
26 void insert()
27 {
28     int item;
29     if (rear == MAX_SIZE)
30         printf("\n## Queue Reached Max!!");
31     else
32     {
33         printf("\nEnter The Value to be Inserted : ");
34         scanf("%d", &item);
35         printf("# Position : %d , Inserted Value : %d ", rear + 1, item);

```

```

C > CA1 > C Set5Q2.c > delete_min()
36     arr_queue[rear++] = item;
37 }
38 }
39
40 void delete_min()
41 {
42     if (front == rear)
43         printf("\n## Queue is Empty!");
44     else
45     {
46         int min_element, index = 0;
47         display();
48         for (int i = front; i < rear; i++)
49         {
50             if (arr_queue[i] < arr_queue[index])
51             {
52                 index = i;
53             }
54         }
55         min_element = arr_queue[index];
56         printf("\nMinimum element is %d.\n", min_element);
57         printf("Deleting it\n");
58
59         for(int i=index; i<rear; i++){
60             arr_queue[i] = arr_queue[i+1];
61         }
62         rear = rear - 1;
63         display();
64     }
65 }
66
67 void display()
68 {
69     printf("\n## Queue Size : %d \n", rear);
70     for (int i = front; i < rear; i++)
71         printf("%d ", arr_queue[i]);
72 }

```

Enter the no of queue elements: 5

```

Enter The Value to be Inserted : 32
# Position : 1 , Inserted Value : 32
Enter The Value to be Inserted : 45
# Position : 2 , Inserted Value : 45
Enter The Value to be Inserted : 67
# Position : 3 , Inserted Value : 67
Enter The Value to be Inserted : 12
# Position : 4 , Inserted Value : 12
Enter The Value to be Inserted : 56
# Position : 5 , Inserted Value : 56
## Queue Size : 5
32 45 67 12 56
Minimum element is 12.
Deleting it

```

```

## Queue Size : 4
32 45 67 56 [crypticani@fedora DSA_Programs]$

```

C > CA1 > C Set5Q3.c > display()

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *next;
8  };
9  struct node *front;
10 struct node *rear;
11
12 void insert();
13 void display();
14
15 void main()
16 {
17     int choice;
18     while(1)
19     {
20         printf("\nQueue operation");
21         printf("\n1.insert an element\n2.Display the");
22         printf("\nEnter your choice: ");
23         scanf("%d", &choice);
24         switch (choice)
25         {
26             case 1:
27                 insert();
28                 break;
29             case 2:
30                 display();
31                 break;
32             case 0:
33                 exit(0);
34                 break;
35             default:
36                 printf("\nEnter valid choice!!\n");
37         }
38     }
```

C > CA1 > C Set5Q3.c > display()

```
37     }
38 }
39
40
41 void insert()
42 {
43     struct node *ptr;
44     int item;
45
46     ptr = (struct node *)malloc(sizeof(struct node))
47     if (ptr == NULL)
48     {
49         printf("\nOVERFLOW\n");
50         return;
51     }
52     else
53     {
54         printf("\nEnter value: ");
55         scanf("%d", &item);
56         ptr->data = item;
57         if (front == NULL) //inserting node in empty
58         {
59             front = ptr;
60             rear = ptr;
61             front->next = NULL;
62             rear->next = NULL;
63         }
64         else //inserting node next to the previous n
65         {
66             rear->next = ptr;
67             rear = ptr;
68             rear->next = NULL;
69         }
70         printf("Inserted!\n");
71     }
72 }
73
74 void display()
```

C > CA1 > C Set5Q3.c > 📄 display()

```
58     {
59         front = ptr;
60         rear = ptr;
61         front->next = NULL;
62         rear->next = NULL;
63     }
64     else //inserting node next to the previous node
65     {
66         rear->next = ptr;
67         rear = ptr;
68         rear->next = NULL;
69     }
70     printf("Inserted!\n");
71 }
72 }
73
74 void display()
75 {
76     struct node *ptr;
77     ptr = front;
78     if (front == NULL)
79         printf("\nEmpty queue\n");
80     else
81     {
82         printf("\nValues inside Queue are: ");
83         while (ptr != NULL)
84         {
85             printf("%d ", ptr->data);
86             ptr = ptr->next;
87         }
88         printf("\n");
89     }
90 }
```

Queue operation

```
1.insert an element
2.Display the queue
0.Exit
```

Enter your choice: 1

```
Enter value: 43
Inserted!
```

Queue operation

```
1.insert an element
2.Display the queue
0.Exit
```

Enter your choice: 1

```
Enter value: 46
Inserted!
```

Queue operation

```
1.insert an element
2.Display the queue
0.Exit
```

Enter your choice: 2

Values inside Queue are: 43 46

Queue operation

```
1.insert an element
2.Display the queue
0.Exit
```

Enter your choice: 0

[crypticani@fedora DSA_Programs]\$